

7. Multiple Index Usage & Query Planner

Objective

In this topic, we'll answer the following questions:

- [What Is a Multiple Index?](#)
- [How Does PostgreSQL Combine Multiple Indexes? \(Bitmap Index Scan\)](#)
- [Why Does PostgreSQL Usually Use Only One Index Per Table?](#)
- [How Does the PostgreSQL Query Execution Flow?](#)
- [How Does the PostgreSQL Query Planner Choose an Execution Plan?](#)

1. What is Multiple Index?

Multiple indexes are two or more independent indexes defined on the same table. Each index provides an alternative access path, allowing the query planner to choose the most efficient strategy for executing a query.

```
CREATE INDEX idx_users_email
ON users(email);

CREATE INDEX idx_users_name
ON users(name);
```

Why Multiple Indexes Matter

Multiple indexes improve query optimization by providing alternative access paths to the same table for different query patterns. Their presence enables the query planner to evaluate several execution strategies, such as index scans, bitmap scans, or sequential scans, and to estimate the cost of each based on table statistics, data distribution, and selectivity.

Advantages:

- Faster SELECT queries.
- Better query optimization.
- Supports different search conditions.
- Improves performance for multi-column queries.

Disadvantages:

- Slower INSERT, UPDATE, and DELETE operations.
- Uses more storage space.
- Increases maintenance overhead.
- Too many indexes can reduce efficiency.

2. How Does PostgreSQL Combine Multiple Indexes?

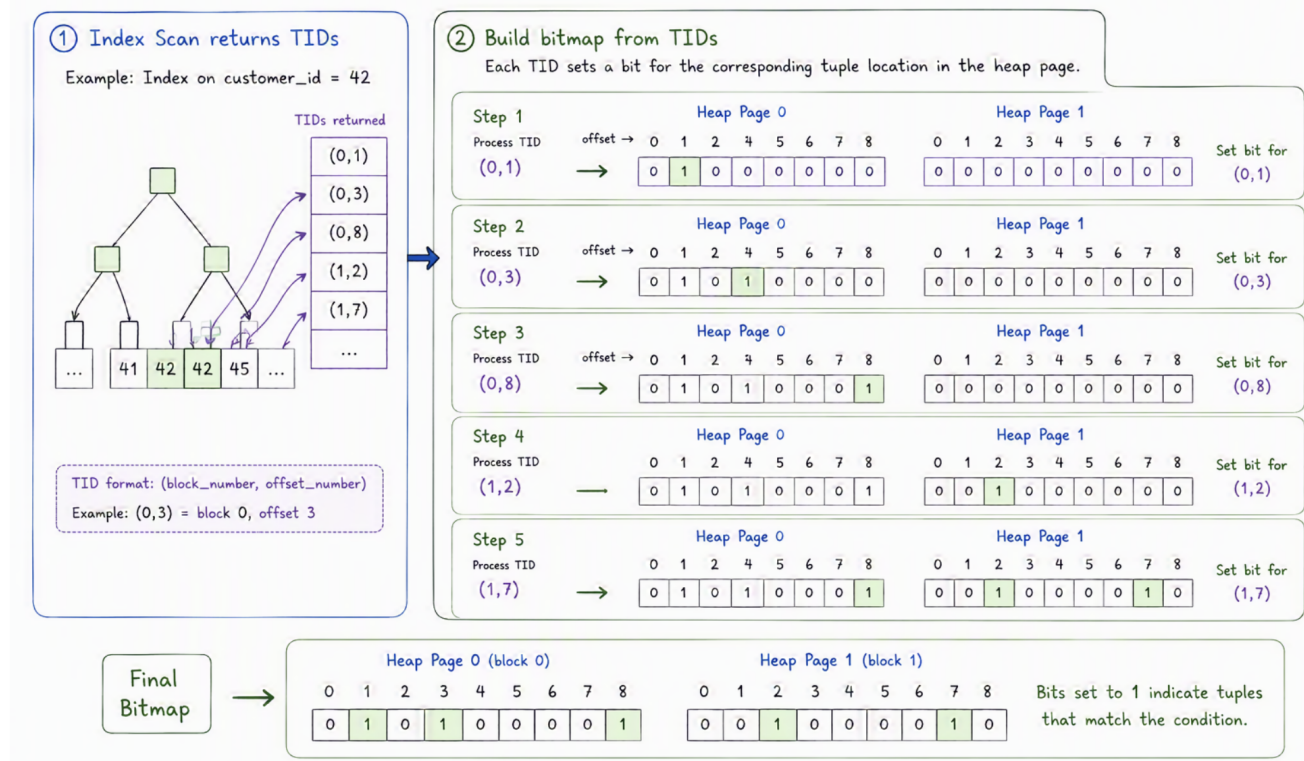
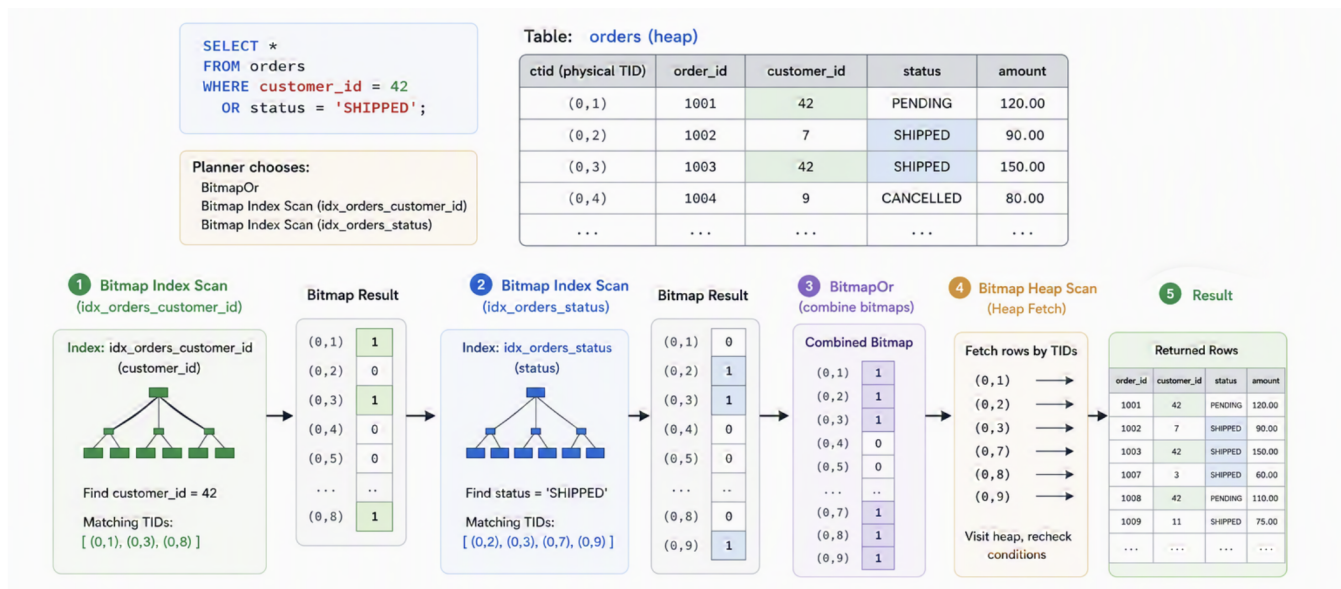
If an index scan or a seq scan aren't the perfect option, Postgres can use the the bitmap index scan as a kind of hybrid approach. It is typically chosen when a query matches too many rows for an regular index scan, but not so many that a sequential scan would be the best option.

This shows up in an EXPLAIN plan as a two-phased approach.

1. **Bitmap Index Scan:** First, Postgres scans one or more indexes to create an in-memory "bitmap", a simple map of all the table pages that *might* contain rows you need.
2. **Bitmap Heap Scan:** The bitmap is used to visit the main table. The key here is that it reads the required pages from the disk sequentially, which can be much faster than the random jumping of a standard index scan.

PostgreSQL combines multiple indexes using a mechanism called:

Bitmap Index Scan + Bitmap Heap Scan



Advantages

1. Better selectivity
2. Avoids creating many composite indexes
3. Supports flexible queries
4. Bitmap scans reduce random I/O

Disadvantages

1. More CPU overhead
2. Not as fast as a good composite index
3. Heap access is still required
4. Planner may choose only one index
5. More indexes increase write cost

3. Why Does PostgreSQL Usually Use Only One Index Per Table?

When executing a query, PostgreSQL's Planner (Optimizer) evaluates multiple execution strategies and selects the one with the lowest estimated cost. One of these decisions is whether to use a **single index**, **multiple indexes**, or perform a **Sequential Scan**. Contrary to a common misconception, PostgreSQL is capable of using multiple indexes on the same table. However, it does not do so by default. The planner chooses multiple indexes only when the estimated cost of combining them is lower than the cost of alternative execution plans. The decision is entirely cost-based and relies on table statistics, index statistics, selectivity estimates, and I/O cost estimates.

When the Planner Chooses Multiple Indexes:

1. Multiple Indexed Conditions

If a query contains multiple predicates and each predicate has its own index, the planner considers combining those indexes.

Example:

```
SELECT *
FROM employees
WHERE department = 'IT'
AND salary > 5000;
```

If both department and salary are indexed, PostgreSQL may use both indexes through a **Bitmap Index Scan**.

2. High Combined Selectivity

Each individual condition may return a relatively large number of rows, while their combination significantly reduces the result set.

For example:

- department = 'IT' returns 30% of the table.
- salary > 5000 returns 20% of the table.
- Both conditions together return only 2%.

In this case, combining both indexes can be more efficient than using either index alone.

3. Lower Estimated Cost

The planner estimates the total cost of combining indexes, including:

- Scanning each index
- Combining bitmap results (Bitmap AND / Bitmap OR)
- Accessing the required table pages

If this total estimated cost is lower than the alternatives, multiple indexes are selected.

4. AND / OR Predicates

Multiple indexes are commonly used when queries contain logical operators such as:

- AND Bitmap AND
- OR Bitmap OR

These operations allow PostgreSQL to combine the results produced by multiple indexes.

When the Planner Does Not Choose Multiple Indexes:

The planner avoids using multiple indexes when another execution strategy is estimated to be less expensive.

1. A Single Index Is Sufficient

If one index already identifies a very small number of rows, combining additional indexes provides little or no benefit.

Example:

```
SELECT *
FROM employees
WHERE employee_id = 100;
```

Since employee_id is typically a primary key, a single index lookup is sufficient.

2.A Composite Index Exists

If a composite (multi-column) index covers all filtering conditions, the planner usually prefers it over combining separate indexes.

Example:

```
CREATE INDEX idx_emp
ON employees(department, salary);
```

For queries filtering on both columns, a composite index scan is generally more efficient than combining two individual indexes.

3.Low Selectivity

If one of the conditions matches a large portion of the table, the benefit of using its index decreases.

Example:

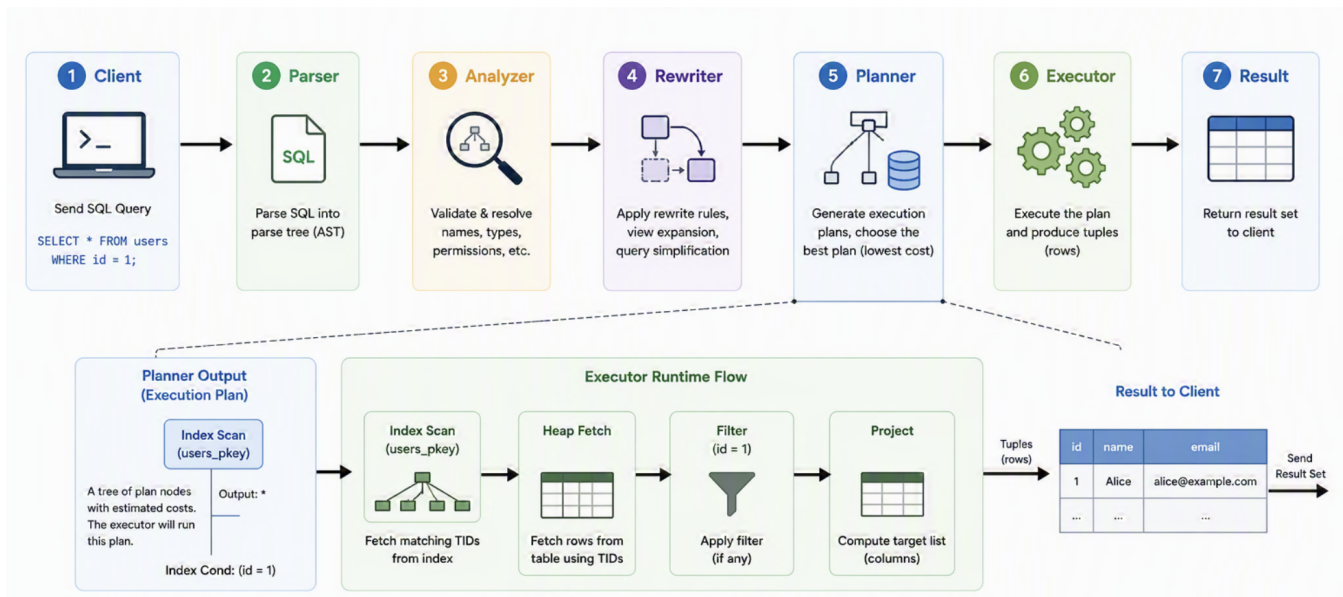
```
WHERE gender = 'M'
```

If approximately half of the rows satisfy the condition, a Sequential Scan may be more efficient than using an index.

4.Small Tables

For small tables, sequential scanning is often faster than index access because the overhead of traversing an index outweighs its benefits.

4. How Does the PostgreSQL Query Execution Flow?



1.Client

Responsibility: The component that sends the SQL query to the database.

- Receives the SQL query from the user or application.
- Sends the query to the database server.

- Receives the query result and returns it to the user.

2. Parser

Responsibility: Checks whether the SQL query follows the correct syntax.

- Validates SQL syntax.
- Detects syntax errors.
- Builds a parse tree representing the query structure.

3. Analyzer (Semantic Analyzer)

Responsibility: Verifies that the query is semantically correct.

- Checks whether referenced tables exist.
- Verifies that referenced columns exist.
- Validates user permissions.
- Ensures data types are compatible.
- Resolves object names and references.

4. Rewriter (Query Rewriter)

Responsibility: Rewrites the query into an equivalent but more optimized logical form.

- Expands views into their underlying queries.
- Simplifies query expressions.
- Rewrites subqueries into more efficient forms when possible.
- Applies logical query transformations.

5. Planner (Optimizer)

Responsibility: Determines the most efficient execution strategy for the query.

- Generates alternative execution plans.
- Estimates the cost of each plan.
- Chooses the plan with the lowest estimated cost.

6. Executor

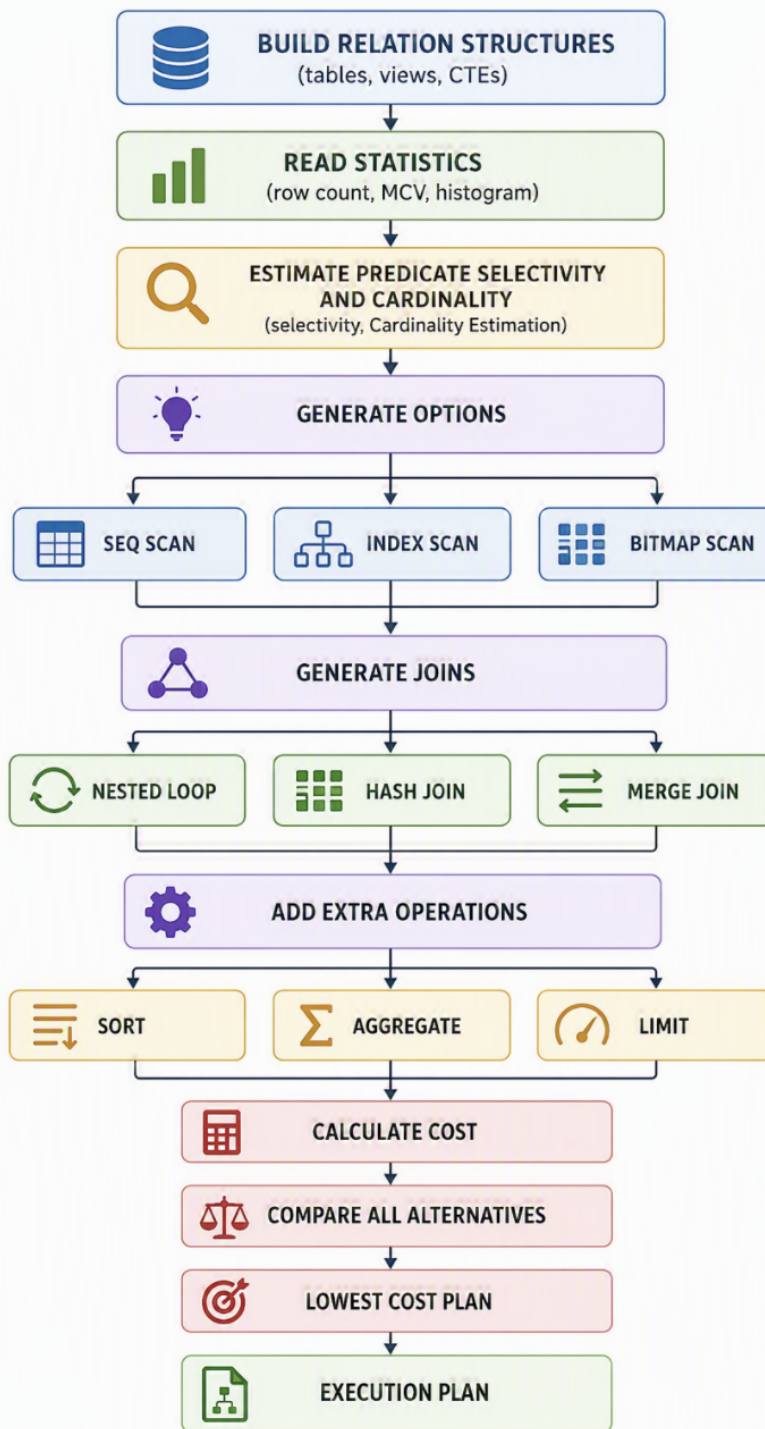
Responsibility: Executes the execution plan selected by the planner and produces the final query result.

- Executes each operator in the execution plan.
- Directly interacts with the storage layer (I/O) to read and write data pages.
- Retrieves data from tables and indexes.

5. How Does the PostgreSQL Query Planner Choose an Execution Plan?

Overview

The PostgreSQL Query Planner is responsible for analyzing SQL queries and selecting the most efficient execution strategy before the query is executed. Instead of directly running a query, PostgreSQL evaluates multiple possible execution approaches and chooses the plan with the lowest estimated cost. The Planner uses query structure, relation metadata, table statistics, available indexes, estimated row counts, and PostgreSQL's cost model to compare different execution alternatives. These alternatives may include different scan methods, join strategies, sorting methods, aggregation approaches, and parallel execution options. The planning process does not measure the real execution time of each possible plan. Instead, PostgreSQL relies on statistical information and cost calculations to predict which strategy will perform best. The accuracy of these predictions depends heavily on the quality of table statistics and database configuration parameters.



1. Build Relation Structures

The Planner begins by building internal optimization structures for every relation participating in the query. Rather than discovering relations, it converts the already parsed and rewritten query into internal representations (RelOptInfo) that will be used throughout the optimization process. These structures contain metadata describing each relation and provide the foundation for access path generation, join planning, and cost estimation. **At this stage, the Planner focuses on understanding the structure of the query rather than evaluating execution costs. The generated relation structures become the basis for every optimization decision made in the following phases.**

Base Tables

Base tables are physical relations that store actual data. The Planner analyzes each table's metadata, available indexes, estimated size, and storage characteristics to prepare possible access strategies. This information is later used when comparing Sequential Scan, Index Scan, Bitmap Heap Scan, and other scan methods. **The Planner also records relation-specific information that will be reused throughout planning. This avoids repeatedly collecting metadata during later optimization stages.**

Views

Views have already been expanded during the Query Rewrite phase. The Planner therefore optimizes the underlying base relations as part of the unified query tree rather than treating the view as a separate object. This allows predicates, joins, and other operations to be optimized across the entire query. **Because views become part of the rewritten query, PostgreSQL can optimize them together with the surrounding SQL. This often produces a more efficient execution plan than optimizing the view separately.**

2. Load Statistics

The Planner loads relation statistics to estimate data distribution without scanning the actual tables. These statistics are essential because nearly every optimization decision depends on estimated row counts and value distributions.

Statistics are maintained by the ANALYZE command and stored in PostgreSQL system catalogs. **Using precomputed statistics keeps planning fast while still providing reliable estimates. The quality of these statistics has a direct impact on the quality of the generated execution plan.**

pg_statistic:

*The pg_statistic catalog stores summarized statistics for table columns. Instead of reading table data, the Planner consults these statistics to estimate query costs and result sizes. **The catalog contains statistical summaries rather than actual data values. This allows PostgreSQL to estimate query behavior without scanning the underlying tables.***

NDISTINCT

NDISTINCT represents the estimated number of distinct values in a column. It helps estimate filter selectivity and join cardinality by predicting how many unique values exist. **It is especially important for estimating GROUP BY, DISTINCT, and join operations. More accurate NDISTINCT values generally lead to better cardinality estimates.**

Most Common Values (MCV)

The MCV list stores the values that occur most frequently within a column. Using these frequencies allows the Planner to avoid assuming uniform data distribution, improving estimates for equality predicates. **This is particularly useful for columns with skewed data distributions. Without the MCV list, PostgreSQL might significantly misestimate equality conditions.**

Histogram

Histograms describe the overall distribution of column values. They are primarily used to estimate selectivity for range predicates such as <, >, BETWEEN, and similar comparisons. **Histogram boundaries help the Planner approximate how data is distributed across a column. This improves estimates for range-based filtering conditions.**

3. Estimate Predicate Selectivity and Cardinality

The Planner estimates how filter conditions reduce the number of rows flowing through the execution plan. These estimates determine the expected size of intermediate results before execution begins. **This phase has a major influence on the overall execution plan because nearly every later optimization decision depends on these estimates. Even small estimation errors can affect join selection and access path choices.**

Selectivity

Selectivity represents the fraction of rows expected to satisfy a predicate. Highly selective predicates usually favor index access, while low selectivity often makes sequential scans more efficient. **The Planner calculates selectivity using available statistics such as MCV lists, histograms, and NDISTINCT values. Different predicate types use different estimation methods.**

Cardinality Estimation

Cardinality estimation predicts the number of rows produced after each operation. These estimates strongly influence access path selection, join ordering, join algorithms, and overall plan cost. Incorrect cardinality estimates are one of the most common causes of inefficient execution plans. **Each estimated row count becomes the input for the next optimization step. Improving cardinality estimation is one of the most important factors in producing efficient execution plans.**

4. Generate Access Paths

For each base relation, the Planner generates multiple candidate access paths. At this stage, PostgreSQL does not choose a scan method. Instead, it builds every reasonable alternative so they can later be compared using the cost model.

The purpose of this phase is to explore different ways of retrieving data from the same relation. Each generated path contains estimated costs and row information that will be evaluated during plan selection.

Sequential Scan

Reads the entire table sequentially. It is often preferred when a large portion of the table must be read or when index usage would not reduce overall cost. **Sequential Scan can be the most efficient option even for indexed tables when PostgreSQL expects that many rows will be returned. The Planner compares the cost of reading all pages sequentially against the random access cost of using an index.**

Index Scan

Uses an index to locate qualifying rows. The Planner evaluates index selectivity, table size, and estimated random page access costs before considering this path. **Index Scan is generally preferred when the expected number of matching rows is small. The Planner also considers whether fetching rows from the heap will introduce significant random I/O overhead.**

Index Only Scan

Returns data directly from the index whenever all required columns are available and the visibility map allows heap access to be skipped. This can significantly reduce disk I/O. **The effectiveness of Index Only Scan depends on how frequently PostgreSQL can confirm row visibility without accessing the table heap. Well-maintained visibility maps improve the chance of avoiding additional page reads.**

Bitmap Heap Scan

Builds a bitmap from index entries before reading heap pages. This approach reduces random I/O and is often selected for medium-sized result sets where a regular Index Scan would require many scattered page accesses. **Bitmap Heap Scan provides a middle ground between Sequential Scan and Index Scan. It is useful when many rows match a condition but reading the entire table would still be unnecessary.**

5. Enumerate Join Paths

When multiple relations are involved, the Planner generates different join orders and join algorithms. For smaller queries, PostgreSQL explores many possible join combinations exhaustively. For larger join graphs, it may switch to the Genetic Query Optimizer (GEO) to reduce planning time. **Join ordering is one of the most important optimization decisions because different join sequences can produce significantly different execution costs. The Planner evaluates possible alternatives before selecting the cheapest join strategy.**

Nested Loop Join

Scans one relation while repeatedly searching another relation for matching rows. It performs well when the outer relation is small or when indexes support efficient lookups. **Nested Loop is especially effective for highly selective queries where the inner relation can be accessed quickly. However, performance can degrade when the outer relation produces many rows.**

Hash Join

Builds a hash table for one relation and probes it using rows from the other relation. It is commonly chosen for large equality joins when sufficient memory is available. **The Planner estimates whether the hash table can fit into available memory. If the estimated size is too large, additional disk operations may increase the cost of this join method.**

Merge Join

Processes two sorted inputs simultaneously. It is efficient when both inputs are already sorted or when sorting costs are relatively low. **Merge Join can avoid additional sorting when suitable indexes already provide ordered input. It is commonly considered for large joins where both relations contain many rows.**

6. Generate Upper-Level Paths

After access paths and join paths have been generated, the Planner creates additional path nodes required to satisfy the complete query. These operations extend existing paths rather than replacing them. **Upper-level paths represent operations applied after the main relation access and join decisions. The Planner evaluates different combinations of these operations before creating the final execution plan.**

Sort

Adds ordering required for ORDER BY clauses or operators that require sorted input. **The Planner considers both in-memory sorting and external disk-based sorting costs. The estimated data size and available work memory affect whether sorting is considered expensive.**

Aggregate

Generates aggregation paths for GROUP BY and aggregate functions. The Planner may choose different aggregation strategies depending on estimated data volume and available memory. **PostgreSQL may choose between Hash Aggregate and Group Aggregate depending on the expected number of groups, input ordering, and memory requirements.**

7. Estimate Costs

The Planner assigns estimated costs to every candidate path. These estimates are based on PostgreSQL's cost model rather than actual execution time. **Cost estimation allows PostgreSQL to compare different execution strategies before running the query. The Planner uses mathematical estimates based on statistics, configuration parameters, and expected resource usage.** The cost model does not measure real execution speed. Instead, it predicts the relative expense of each possible plan and selects the alternative with the lowest estimated cost.

Startup Cost

Represents the estimated work required before the first row can be returned. This metric is especially important for queries containing LIMIT. **Startup cost becomes a key factor when PostgreSQL needs to return the first rows as quickly as possible. Plans with low startup cost may be preferred even if their total execution cost is higher.** For example, an Index Scan may have a higher total cost than a Sequential Scan but still be selected for a query with a small LIMIT because it can produce initial rows faster.

Total Cost

Represents the estimated work required to complete the entire operation. This is the primary metric used when comparing competing execution paths. **Total cost includes the expected cost of processing all rows, reading required pages, and performing additional operations such as sorting or joining. The Planner uses this value to compare complete execution strategies.** A plan with the lowest total cost is not always selected if another plan provides a significantly lower startup cost and the query does not require processing the complete result set.

CPU Cost

Estimates processor work required for tuple processing, expression evaluation, filtering, and comparisons. **CPU cost calculations include operations such as evaluating WHERE conditions, processing tuples, and executing expressions. PostgreSQL uses configurable parameters such as `cpu_tuple_cost` and `cpu_operator_cost` when estimating these operations.** CPU cost becomes more important when queries perform complex expressions, functions, calculations, or process large numbers of rows.

Disk I/O Cost

Estimates storage access costs, accounting for both sequential and random page reads. **The Planner distinguishes between sequential and random page access because they have different performance characteristics. Parameters such as `seq_page_cost` and `random_page_cost` influence these calculations.** For large table scans, sequential I/O is usually considered cheaper, while index-based access may introduce higher random I/O costs.

Parallel Cost

Represents the additional overhead introduced by parallel workers, including worker startup, communication, and result gathering. **The Planner compares the expected performance improvement from parallel execution against the additional coordination overhead. Parallel execution is selected only when the estimated benefit exceeds the extra cost.**

8. Select the Cheapest Path

After all candidate paths have been generated and costed, the Planner compares them and selects the path with the lowest estimated cost. This selected path becomes the logical foundation of the final execution plan. **At this stage, PostgreSQL has already evaluated different scan methods, join strategies, and query operation combinations. The Planner keeps only the most efficient path according to its cost model.** The selected path represents the optimal strategy according to PostgreSQL's estimates. It does not guarantee the fastest real execution time, because the decision depends on the accuracy of statistics and cost parameters.

9. Build the Final Plan Tree

The final optimization step converts the selected Path tree into a Plan tree. This hierarchical structure contains scan nodes, join nodes, sort nodes, aggregate nodes, and all other execution operators required by the Executor. **The Path structure is an optimization representation, while the Plan tree is the executable representation used by PostgreSQL. During this conversion, PostgreSQL creates the actual executor nodes that will process the query.** The completed Plan tree represents PostgreSQL's chosen physical execution strategy and is passed directly to the Executor for execution. **Each node in the Plan tree defines a specific operation and its relationship with other operations. The Executor follows this tree structure from the top level down to retrieve and process the final result.**